Flight Delay Prediction via Gradient Boosting and Temporal Deep Learning
An Offline Rescheduling Analysis

Samiksha Jangid
Data Pipeline & Leakage Prevention
Team T022, Section C

Rashmi Anand
Exploratory Analysis & Feature Engineering
Team T022, Section C

Ankit Raj Singh
Logistic Regression Baseline
Team T022, Section C

Shubhaang Kataruka
XGBoost & Hyperparameter Tuning
Team T022, Section C

Lakshya
Evaluation & Calibration
Team T022, Section C

***Abstract*—We predict whether a scheduled flight will arrive at least fifteen minutes late, using schedule fields and strictly prior-date histories under an explicit, unverified outcome-availability assumption, and study whether the resulting probabilities can support proactive rescheduling. Using one year of 2013 US flights originating from the three major New York airports, we build a leakage-aware pipeline whose safeguards operate at three levels: a pre-departure feature allowlist, historical delay statistics aggregated from prior calendar dates only, and a strictly chronological train/validation/test split. The frozen 31-feature evaluation uses temporal (cyclic), route and prior-date historical features. A separate experiment adds two schedule-based airport-load proxies and evaluates them on validation only; that 33-feature variant has no held-out test score. We train a class-balanced Logistic Regression and a tuned XGBoost classifier. Probabilities are calibrated on a held-out late-training window (isotonic regression, selected by validation Brier score), and a single operating threshold per model is frozen on validation before frozen evaluation on the held-out test set. This calendar test period was also used in an earlier pre-congestion experiment and later reproduced for audit, so it was not untouched throughout development. XGBoost is the stronger descriptive held-out candidate (test Average Precision 0.337, ROC-AUC 0.599) but absolute discrimination is modest, and a proper-score audit shows the calibrated probabilities are not convincingly better than a prevalence baseline. We then derive an offline cost-sensitive rescheduling rule with a closed-form net-benefit and a cost-optimal action threshold $\tau^\star = C_a/(r\,C_d)$, and report a sensitivity analysis: a model-guided rule beats inaction only when interventions are cheap relative to a delay, and XGBoost has the more favourable cost-sensitivity profile over the evaluated cost range. In Phase 2, calibrated XGBoost, the embedding MLP and the XGBoost–MLP stack achieved validation AP of 0.2129, 0.2082 and 0.2093, respectively. The stack's Brier score was 0.137145, compared with 0.137199 for calibrated XGBoost. The offline decision analysis uses the calibrated classical models.**

***Index Terms*—flight delay prediction, data leakage, probability calibration, gradient boosting, entity embeddings, cost-sensitive decision analysis, decision curve analysis.**

I. Introduction

Flight delays impose large and cascading costs on passengers, carriers and the air-traffic system. Most of the operational value of a delay prediction is realised *before* the aircraft leaves the gate, when swaps, buffers and re-bookings are still possible. This motivates a strict problem definition: a prediction made at the scheduled-departure decision point may use only information that is genuinely available at that moment.

The central methodological risk in such a task is *data leakage*—allowing information unavailable at prediction time to influence training or evaluation, which inflates apparent accuracy and produces a model that fails in deployment. A second risk is *miscalibration*: a classifier may rank flights well yet emit probabilities that do not correspond to real-world frequencies, which is fatal for any downstream decision rule that thresholds a probability.

This report covers both phases of the project. First, it describes a data pipeline with three independent defences (Section VI). Second, documented and unit-tested feature engineering, including a cyclic time encoding and prior-date historical aggregates (Section V). Third, a calibrated chronological evaluation of a Logistic Regression baseline against a tuned XGBoost model, with frozen held-out results and explicit test-period reuse disclosure (Section VII). Fourth, an offline rescheduling what-if analysis built on explicit cost assumptions, including the cost-optimal operating threshold and a sensitivity study (Section VIII). Fifth, it evaluates the embedding MLP and an ML+DL stack on aligned validation flights (Section IX). Throughout, we favour transparent reporting of weaknesses over optimistic claims.

II. Related Work

Flight delay prediction. Delay modelling spans queuing and network propagation models through to data-driven machine learning on large operational records. The US Bureau of Transportation Statistics (BTS) defines the fifteen-minute threshold for a "delayed" flight and publishes the on-time performance data that underpins much of this literature. Our work adopts the BTS threshold and the pre-departure constraint but deliberately restricts itself to a compact, reproducible single-year dataset for the baseline phase.

Gradient boosting. Gradient-boosted decision trees, and XGBoost in particular, are strong tabular learners that capture non-linear feature interactions with modest tuning, making them a natural comparator to a linear baseline.

Probability calibration. Guo *et al.* show that modern classifiers are frequently miscalibrated and that post-hoc recalibration is often necessary. Classic post-hoc methods include Platt scaling (a sigmoid fit) and isotonic regression (a non-parametric monotone map); Niculescu-Mizil and Caruana compare their behaviour across learners. We evaluate raw, sigmoid and isotonic variants and select by a proper scoring rule.

Evaluation under imbalance. For skewed classes, precision–recall analysis is more informative than ROC alone, because the ROC curve can look optimistic when the negative class dominates. We therefore report scikit-learn Average Precision (AP) as the primary ranking metric, alongside ROC-AUC and the proper scores (Brier, log loss) that also drive our calibration selection. All estimators use the standard scikit-learn and XGBoost implementations.

Decision analysis. Translating probabilities into actions under explicit costs is the subject of decision curve analysis, which plots net benefit against the threshold probability. Our rescheduling analysis generalises this net-benefit view with an explicit recovery factor for partially mitigated delays.

III. Problem Formulation

We treat delay prediction as binary classification. For each flight the target is

$$\texttt{is_delayed} = \mathbf{1}[\text{arrival delay} \geq 15 \text{ min}]\,. \qquad (1)$$

The prediction is made at the scheduled-departure decision point. Consequently, actual departure time, departure delay, actual arrival time, air time and arrival delay are *excluded* from the feature space; only the target is derived from the arrival outcome. Flights whose arrival outcome is undefined (missing arrival delay, e.g. cancellations) are dropped rather than imputed, since their label is not defined. Validation uses a chronological split so that the model is always trained on the past and evaluated on the future. The primary metrics are Average Precision (AP) and ROC-AUC, supported by precision, recall, and the proper scores used for calibration.

IV. Dataset and Exploratory Analysis

The dataset comprises the 2013 flights departing the three major New York airports (EWR, JFK, LGA), the Kaggle starter dataset recommended for the project; the BTS on-time performance data is the larger official extension reserved for later work. After removing flights with an undefined arrival outcome, 327,346 records remain. The overall delay base rate is approximately 26%, but it is strongly seasonal, so the chronological partitions differ markedly in prevalence: training 25.9%, validation 15.6% and test 26.9% (Fig. 1). This shift is a genuine property of the data—the validation window (mid-September to early November) falls in the year's lowest-delay months—and a chronological split *exposes* it rather than hiding it, as a random split would. Because precision–recall metrics depend on positive prevalence, we therefore do not read cross-partition PR differences as improvements (Section VII).

Exploratory analysis, conducted on the training partition only, surfaces the structure the features are designed to exploit (Figs. 2–6):

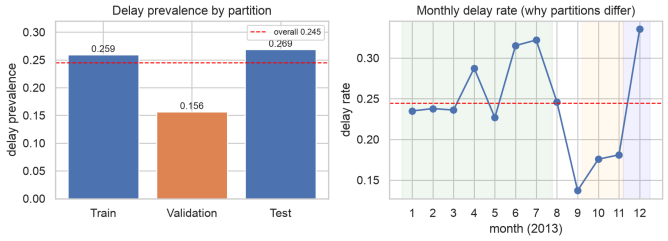


Fig. 1. Delay prevalence by partition (left) and the 2013 monthly delay-rate trend (right). The chronological split places the low-delay autumn months in validation, so partition prevalences differ by design.

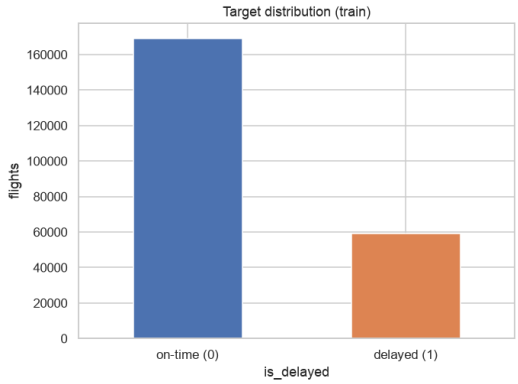


Fig. 2. Target class balance: delayed versus on-time arrivals. The minority (delayed) class motivates Average Precision and class weighting.

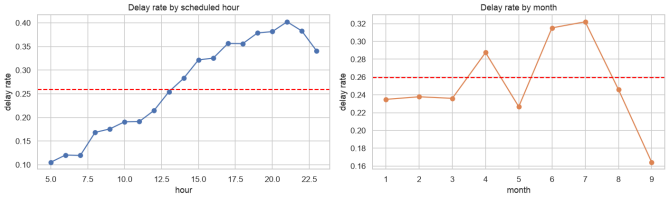


Fig. 3. Delay rate by scheduled hour and month, exposing the strong time-of-day and seasonal structure exploited by the temporal features.

- **Time of day.** Delay risk rises monotonically from roughly 10.5% for early-morning departures to about 40% late in the evening, as delays accumulate and cascade through the day.
- **Seasonality.** Summer is worst (July $\approx 32\%$), September best ($\approx 16\%$), reflecting weather and peak-travel load.
- **Day of week.** Weekdays (27.8%) exceed weekends (20.4%).
- **Carrier.** Delay propensity varies widely by operator, from $\approx 14\%$ to $\approx 41\%$.
- **Origin.** EWR (27.9%) > JFK (25.5%) > LGA (24.0%), and delay risk also varies sharply at the finer route level (Fig. 5).
- **Distance.** Delay rate is largely flat across distance bands and dips only for the longest hauls (Fig. 6), confirming that raw distance is a weak standalone signal and motivating the route and historical features instead.

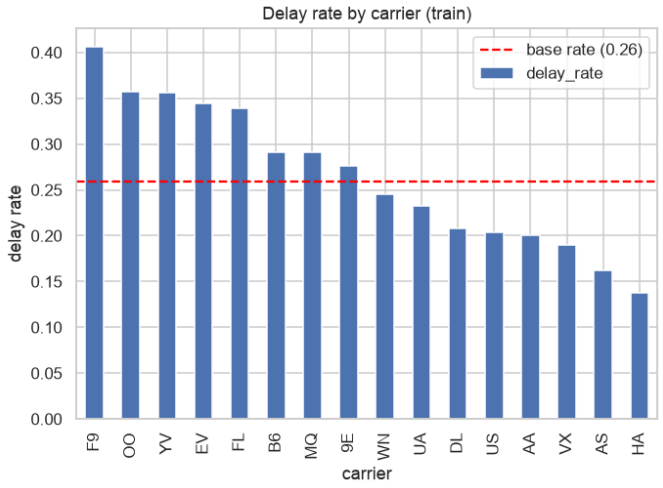


Fig. 4. Delay rate by carrier. Operator-level variation motivates the carrier historical-delay features.

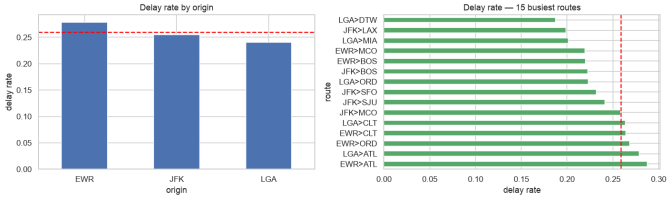


Fig. 5. Delay rate by origin airport (left) and across the fifteen busiest origin→destination routes (right), each against the overall base rate (dashed). Route-level spread of roughly 19% to 29% motivates the route and route-historical features.

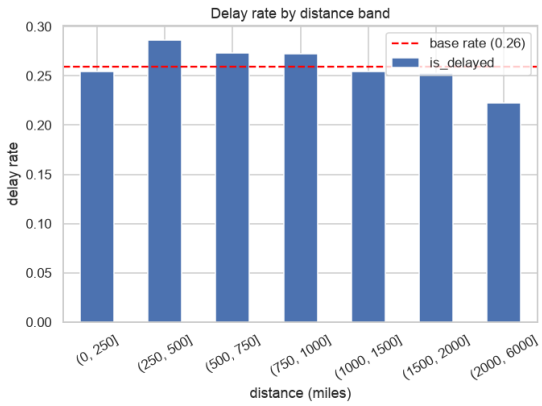


Fig. 6. Delay rate by binned flight distance. The near-flat profile shows distance alone carries little signal, so it enters only as context for the route encoding.

V. Feature Engineering

The canonical frozen model uses 31 features derived from the schedule and historical outcomes on prior calendar dates. Two additional schedule-derived airport-load proxies form a 33-feature variant used only in a validation ablation, not in the held-out test evaluation. The feature engines are documented and unit-tested.

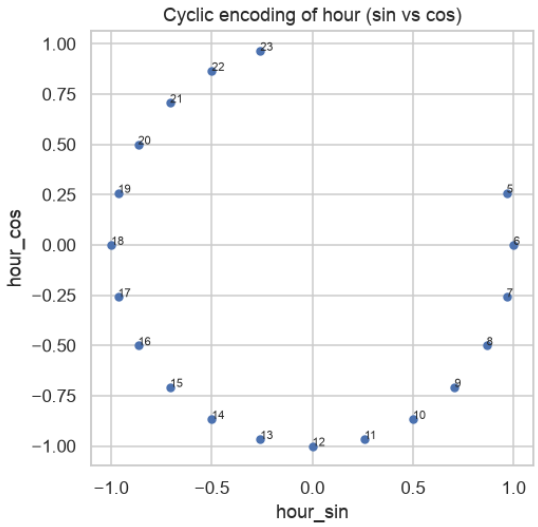


Fig. 7. Cyclic hour encoding: hour 23 sits adjacent to hour 0 on the circle, unlike the raw integer representation.

A. Temporal features

Calendar signals (`day_of_week`, `is_weekend`, `season`) are augmented with a *cyclic* encoding of hour, month and weekday. A raw integer hour misleads a model because it places 23:00 twenty-three units from 00:00, although they are one hour apart. Mapping each periodic value x with period P onto a circle,

$$x \mapsto \left(\sin \tfrac{2\pi x}{P},\ \cos \tfrac{2\pi x}{P}\right), \tag{2}$$

restores the true adjacency (Fig. 7); linear models in particular benefit.

B. Route features

Static route descriptors encode the corridor and its length: the origin→destination identifier, a binned `distance_band`, a long-haul flag ($\geq$ 1500 miles) and $\log(1 + \text{distance})$ to compress the heavy tail (the raw distance signal being weak, Fig. 6). These are fixed regardless of outcome and so introduce no leakage.

C. Historical delay features

For each carrier, origin, destination and route we compute a historical delay rate and flight count from *previous calendar dates only*, using a shifted cumulative aggregation so that same-day and future observations are never included. These prior-date rates correlate with the target and summarise congestion and operational reliability that the static features cannot. Among them, the carrier and route prior-date rates correlate most strongly with the target (Fig. 8), consistent with the operator- and corridor-level structure seen in the exploratory analysis. We document openly that this guarantees prior-date ordering but not the *exact* availability of each outcome at the precise prediction instant; availability-time leakage therefore remains a stated caveat.

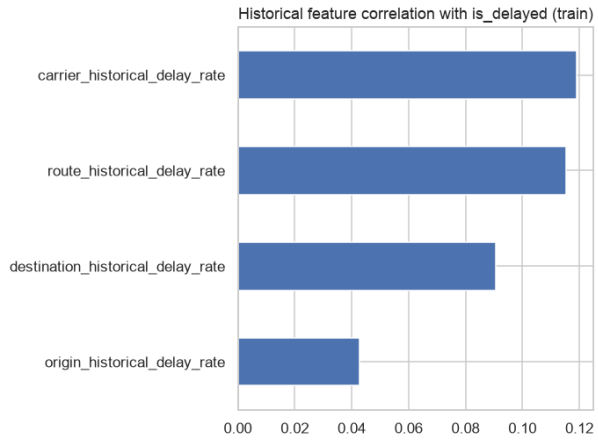


Fig. 8. Pearson correlation of each prior-date historical delay rate with the target on the training partition. Carrier- and route-level histories are the most informative.

D. Airport congestion features

To probe schedule-based airport-load proxies beyond the historical volume features above, we examine two schedule-derived counts: `origin_hourly_departures`, the number of flights scheduled to depart the same origin in the same (date, hour), and `dest_hourly_arrivals`, the analogous scheduled arrival load at the destination. The counts use the NYC-origin timetable, not all airport operations. Destination arrivals are grouped by departure date and scheduled arrival hour; flights crossing midnight are not rolled to the next calendar date. These are schedule-based proxies, not measurements of realized congestion. A validation-only ablation found small marginal differences, but does not establish that congestion is unimportant on a larger or more complete airport dataset. The proxies are excluded from the canonical 31-feature held-out evaluation.

E. The pre-departure allowlist

A single feature contract admits only approved predictors into the design matrix and blocks the target, every post-departure column, and identifiers. The canonical `full` allowlist uses schedule, route and historical columns; the separate `full_congestion` validation variant additionally admits the two congestion proxies. The contract is enforced in code and tests, but prior-date outcome availability at the exact prediction timestamp remains an unverified assumption.

VI. Methodology

A. Chronological split and calibration windows

The data are partitioned by *distinct calendar date* into train/validation/ test at $70/15/15$, so all flights on a date stay together and training dates strictly precede validation dates, which precede test dates. For calibration, the training period is split again in time: the earliest 80% of training dates fit the base estimator and preprocessing (the *base-train* window), and the remaining 20% fit the calibrator (the *calibration* window). No calibration-window label ever fits a base estimator. The resulting windows are summarised in Table I.

TABLE I
Chronological windows (2013).

Window	n	First	Last
Base train	181,405	Jan 1	Jul 23
Calibration	46,894	Jul 24	Sep 12
Validation	50,466	Sep 13	Nov 6
Test (held out)	48,581	Nov 7	Dec 31

B. Models

Two classifiers share the identical training rows and feature contract:

- **Logistic Regression** with `class_weight="balanced"` to counter the class imbalance.
- **Tuned XGBoost** with parameters selected previously by training-only chronological cross-validation: 250 trees, max depth 6, learning rate 0.03, minimum child weight 10, subsample 0.90, column subsample 0.75, L_2 regularisation 10, and `scale_pos_weight` ≈ 2.86 for imbalance.

This XGBoost configuration was selected by `RandomizedSearchCV` over eight sampled candidates on the *training data only*, scored with three expanding chronological folds grouped by whole flight dates and separated by a one-date embargo so that no fold shares a date across the train/validation boundary. The search ranged over tree count, depth, learning rate, minimum child weight, row and column subsampling, L_2 regularisation and positive-class weighting; the held-out test set was not used in this hyperparameter search. The same chronological test period had been evaluated in an earlier pre-congestion experiment, so it was not globally unseen during project development.

C. Calibration

Each model's calibration-window scores are mapped by three candidates—raw (identity), sigmoid (Platt), and isotonic—and the variant minimising validation Brier score (log loss as tie-break) is selected. Calibrators are fit strictly on the calibration window; validation and test labels never fit them.

D. Threshold selection and the frozen protocol

A single operating threshold per model is chosen on validation (maximising F_1; precision-floor variants are reported only as illustrative sensitivity, as no operational cost was approved). The selected calibration map and threshold, the fitted estimators, the dataset hash and the dependency versions are then *frozen* to disk. Only after this does the final stage load the frozen objects and evaluate the held-out test set; it cannot refit or reselect. The published frozen results were subsequently reproduced for an audit without changing any modelling decision. Because this calendar period was also used in an earlier pre-congestion experiment, it must not be described as a test set that was never previously inspected.

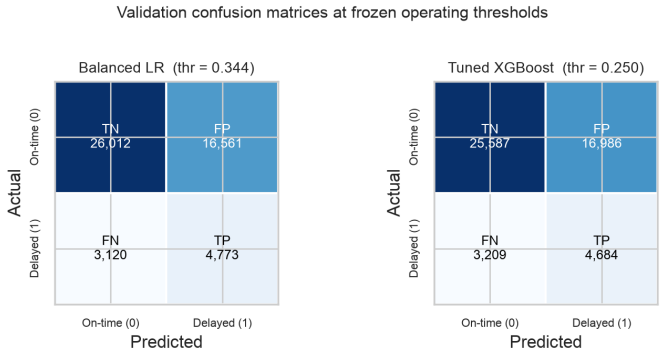


Fig. 9. Validation confusion matrices at the frozen operating thresholds. Both models recover a majority of true delays (high recall) while incurring many false positives, consistent with their low operating precision.

TABLE II
VALIDATION METRICS (ISOTONIC CALIBRATION; FROZEN THRESHOLDS).

Model	Thr.	AP	ROC-AUC	Prec.	Rec.	Brier
Balanced LR	0.344	0.223	0.636	0.224	0.605	0.151
Tuned XGBoost	0.250	0.213	0.628	0.216	0.593	0.137

VII. RESULTS

A. Validation

Table II reports validation performance at the frozen thresholds with the selected (isotonic) calibration for both models. Ranking is similar; both models have low precision ($\approx 22\%$) at their operating points, reflecting the difficulty of the task. ROC and PR curves appear in Figs. 10–11, and the confusion matrices at the frozen thresholds in Fig. 9. On validation alone the balanced LR in fact edges XGBoost on AP and ROC-AUC, but the two differences are tiny and, as the date-block bootstrap below shows, not statistically distinguishable; we therefore defer the model choice to the held-out test (Section VII C), where XGBoost is the stronger candidate on every metric. Both operating points favour recall over precision—catching roughly 60% of true delays at the cost of a large false-alarm count (Fig. 9)—which is the intended behaviour under class weighting and is exactly what the cost analysis of Section VIII later prices.

B. Calibration quality

Isotonic regression minimises validation Brier and log loss for both models and is selected. A proper-score audit, however, tempers any success claim: a constant predictor set to the late-training prevalence attains a Brier of 0.138, which the balanced-LR isotonic probabilities (0.151) *fail* to beat and the XGBoost isotonic probabilities (0.137) beat only narrowly. Improvement over severely miscalibrated raw scores is therefore not sufficient evidence of useful probability forecasts. Table III reports the proper scores and the mean predicted probability for both models before and after calibration: calibration slashes Brier and log loss, yet the calibrated mean prediction (0.23–0.31) still sits above the observed validation prevalence (0.156), i.e. both models over-predict on this lower-prevalence window—another reason the calibrated probabilities are only

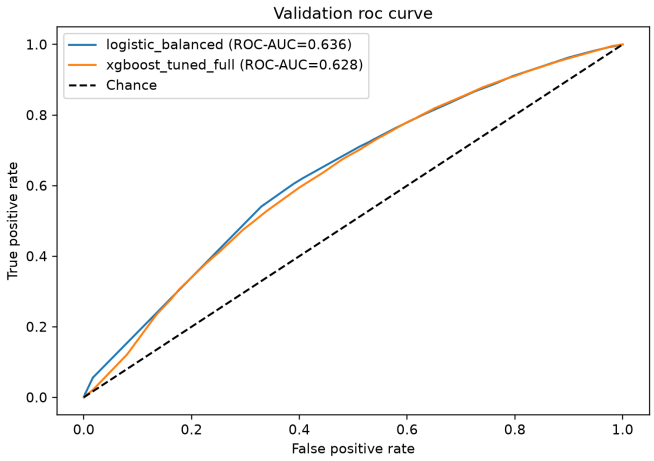


Fig. 10. Validation ROC curves. Discrimination is modest in absolute terms (ROC-AUC ≈ 0.63–0.64).

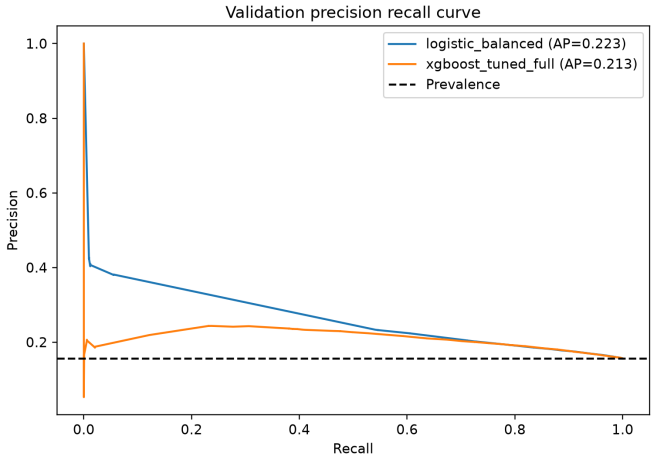


Fig. 11. Validation precision–recall curves against the prevalence no-skill reference.

TABLE III
VALIDATION CALIBRATION QUALITY (OBSERVED PREVALENCE 0.156; PREVALENCE-BASELINE BRIER 0.138). LOWER BRIER/LOG LOSS IS BETTER; A WELL-CALIBRATED MEAN PREDICTION WOULD MATCH THE OBSERVED PREVALENCE.

Model	Variant	Brier	Log loss	Mean pred.
Balanced LR	raw	0.522	1.396	0.782
Balanced LR	isotonic	0.151	0.482	0.305
Tuned XGBoost	raw	0.306	0.860	0.540
Tuned XGBoost	isotonic	**0.137**	**0.443**	0.231

modestly useful. The XGBoost reliability diagram (Fig. 12) is reasonable but not strong.

C. Held-out test

The frozen 31-feature test (Table IV) favours XGBoost on AP, ROC-AUC, precision, recall and F_1. Because prevalence differs between validation (0.156) and test (0.269), the two sets' PR-based metrics are not directly comparable as measures of improvement. A paired date-block bootstrap on validation (300 resamples) gives differences (XGBoost minus LR)

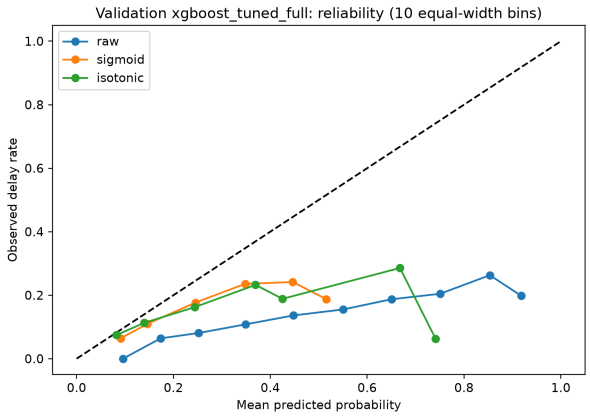


Fig. 12. Validation reliability diagram for the calibrated XGBoost model (ten equal-width bins).

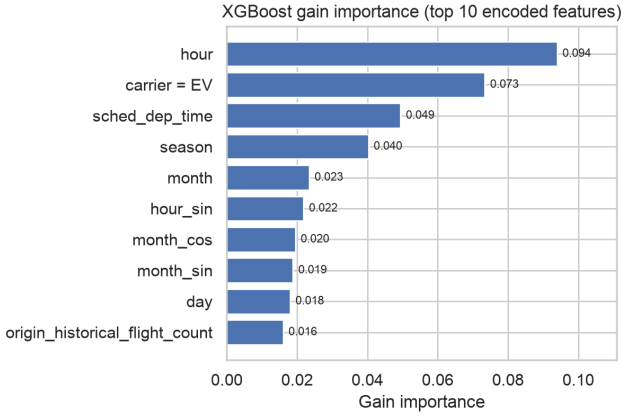


Fig. 13. Gain importance for the ten most influential encoded features in the early-training fit of the frozen XGBoost model. Time-of-day terms dominate, matching the exploratory structure.

TABLE IV
HELD-OUT TEST METRICS FOR THE FROZEN 31-FEATURE MODEL.

Model	AP	ROC-AUC	Prec.	Rec.	F_1
Balanced LR	0.294	0.544	0.296	0.488	0.369
Tuned XGBoost	**0.337**	**0.599**	**0.335**	**0.532**	**0.411**

whose 95% intervals include zero (e.g. AP $[-0.027, 0.011]$), so the ranking advantage is descriptive rather than statistically established. The test period was also used for an earlier pre-congestion experiment and was later replayed to verify the audit; it was not untouched across the full development history.

D. Feature importance

The tuned XGBoost model's gain-importance ranking (Fig. 13) is dominated by time-of-day signals: scheduled `hour`, its cyclic `hour_sin` encoding and `sched_dep_time` account for the largest share, followed by the `season` flag and a high-delay carrier indicator. This agrees with the exploratory finding that delay risk is primarily driven by when a flight departs (Figs. 3), and confirms that the engineered temporal features carry real model weight rather than mere decoration. In the validation-only ablation, adding these two features changed XGBoost AP from 0.2112 to 0.2107 and balanced-LR AP from 0.2532 to 0.2533. These small differences do not establish that airport congestion is unimportant. Importance here is model-relative and does not imply causation; correlated calendar features can share or dilute one another's attributed gain.

VIII. Offline Rescheduling What-If Analysis

A. Cost model

We ask, retrospectively, what the net operational cost would have been had a proactive rescheduling action been taken on flights whose calibrated delay probability exceeds a threshold. Per flight we assume homogeneous costs: C_d for an unmitigated delay, C_a for taking the action (paid whenever we act), and a recovery fraction $r \in [0, 1]$ of the delay cost avoided when we act on a flight that truly delays. The realised cost by decision a and outcome y is

	$y = 0$ (on time)	$y = 1$ (delayed)
$a = 0$ (no action)	0	C_d
$a = 1$ (action)	C_a	$C_a + (1 - r)C_d$

These assumptions—homogeneous per-flight costs, independent flights, a single binary action, and no network feedback—are strong and are declared as limitations; no operational cost has been approved, so all quantities are expressed relative to C_d.

B. Closed-form net benefit and the optimal threshold

For a policy that acts on tp true delays and fp on-time flights, the saving relative to the no-action status quo is

$$\text{net} = \text{tp}\,(r\,C_d - C_a) - \text{fp}\,C_a, \tag{3}$$

a cost-sensitive generalisation of decision-curve net benefit. Acting on a flight has non-negative expected value exactly when its calibrated probability p satisfies $p\,r\,C_d \geq C_a$, giving the *cost-optimal action threshold*

$$\tau^\star = \frac{C_a}{r\,C_d} = \frac{C_a/C_d}{r}. \tag{4}$$

The decision boundary is thus read directly from the cost assumptions—which is precisely why calibration matters operationally, as only a well-calibrated p makes $\tau^\star$ the right place to cut.

C. Sensitivity and findings

Because real costs are unknown, we sweep the cost ratio C_a/C_d at $r = 0.8$ on the validation partition (preserving the held-out test), evaluating each model at its optimal $\tau^\star$ (Table V, Fig. 14).

Three findings follow. First, a model-driven rule beats inaction only when the action is cheap relative to a delay—up to $C_a/C_d \approx 0.10$ for Logistic Regression and ≈ 0.15 for XGBoost; beyond that, low precision makes the false-alarm term in (3) dominate and intervention is not justified. Second, **XGBoost has the more favourable sensitivity profile across the meaningful cost range**—a wider contiguous positive-saving

TABLE V
RESCHEDULING SENSITIVITY: NET SAVING PER FLIGHT (UNITS OF C_d) AT THE COST-OPTIMAL THRESHOLD, $r = 0.8$. "+" BEATS NO ACTION.

C_a/C_d	$\tau^\star$	Balanced LR	Tuned XGBoost
0.05	0.063	+0.075	+0.075
0.10	0.125	+0.025	+0.034
0.15	0.188	−0.016	+0.005
0.20	0.250	−0.049	−0.012
0.25	0.313	−0.056	−0.018

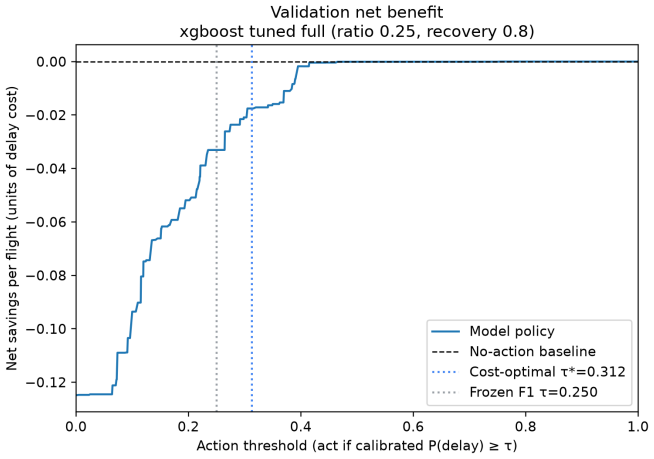


Fig. 14. Validation XGBoost net saving per flight versus action threshold (cost ratio 0.25 and recovery 0.8). The curve crosses the no-action line, and the cost-optimal $\tau^\star$ moves with the assumed cost ratio.

region (profitable up to $C_a/C_d \approx 0.15$ versus ≈ 0.10 for Logistic Regression) and higher oracle efficiency at matched cost ratios—a decision-relevant advantage the ranking metrics alone did not make vivid. The single exception is a degenerate high-cost point ($C_a/C_d = 0.30$), where Logistic Regression records a negligible positive saving only because its calibrated scores rarely clear the high threshold and it therefore acts on almost no flights; this is not a meaningful reversal. Third, even the best attainable efficiency is modest (about 64% of the perfect-foresight oracle at the cheapest ratio), bounding what any policy built on the current probabilities can deliver.

IX. DEEP LEARNING EXTENSION (PHASE 2)

Phase 2 adds a compact neural model to test whether learned representations of airline and airport identity capture interactions that the engineered tabular features miss. It reuses the identical target, chronological partitions, and pre-departure feature allowlist as the Phase 1 pipeline; the held-out test partition is never used for training, early stopping, or model selection.

A. Architecture

The model is an embedding multilayer perceptron. Carrier, origin, and destination are mapped to trainable entity embeddings [7]; the remaining standardised numeric features and the route/distance one-hot blocks are concatenated with the embeddings and passed through dense layers with ReLU activations and dropout, producing a single delay logit per flight. Reserved indices for missing (0) and unseen (1) categories let the model score airports and carriers absent from the training period without errors. Cyclic calendar inputs encode scheduled time, and the MLP operates on one flight per row.

B. Training and reproducibility

The network is trained with `BCEWithLogitsLoss` and the AdamW optimiser. Early stopping monitors validation Average Precision (AP) with a fixed patience, and the best checkpoint is restored. Python, NumPy, and PyTorch seeds are fixed and deterministic algorithms requested. Repeated CPU runs in the verification environment reproduced the exported probabilities bit-for-bit. Hyperparameters are tuned by a seed-controlled search over learning rate, batch size, hidden widths, dropout, and weight decay, selected on validation AP alone.

C. Regularisation and ablation

The original search varied hyperparameters and seeds. A separate ablation compared all six configurations at the same three seeds (42, 46 and 50). Each result is paired against the baseline at the corresponding seed. Lowering the learning rate improved AP by 0.004759 on average. Raising dropout to 0.3 improved it by 0.002135 on average. The matched-seed results are reported separately from checkpoint selection.

D. Hybrid approach

We implement a stacked ensemble with a logistic meta-learner over the frozen XGBoost and MLP probabilities, fitted only on the calibration partition and evaluated on validation. The stack does *not* improve ranking over the best single model. The XGBoost and MLP residuals have correlation 0.968, indicating substantial overlap in their validation errors. The result is consistent with work showing that gradient-boosted trees can be strong on feature-engineered tabular problems [8], [9]. Representation-level fusion through gradient-boosted leaf features is one possible follow-up [10].

E. Fair comparison protocol

The embedding MLP exports probabilities on the same calibration and validation rows, with identical flight identifiers, dates, and labels, in the project prediction schema. Calibration is fitted on 46,894 late-training flights. Table VI reports scores on the same 50,466 validation flights. AP and trapezoidal PR area are reported separately. The 25% precision floor was attainable for calibrated Logistic Regression (5.56% recall) and unattainable for calibrated XGBoost, calibrated MLP and the hybrid. The exported precision, recall, F1 and confusion counts use each model's validation maximum F1 threshold. These operating points describe validation performance.

F. Schedule and route inputs

We also compare the models using only the 23 schedule and route predictors. This comparison uses the same flights and omits all eight outcome-history predictors, whose availability at the decision timestamp is unverified. With these inputs, AP was 0.2217 for calibrated Logistic Regression, 0.2116 for calibrated XGBoost, 0.2075 for the calibrated MLP, and 0.2080

TABLE VI
PHASE 2 VALIDATION ON 50,466 ALIGNED FLIGHTS.

Model	AP	PR AUC	ROC-AUC	Brier
Logistic (iso.)	.2233	.2641	.6360	.150899
XGBoost (iso.)	.2129	.2114	.6279	.137199
MLP (iso.)	.2082	.2039	.6207	.137476
Hybrid	.2093	.2092	.6255	.137145

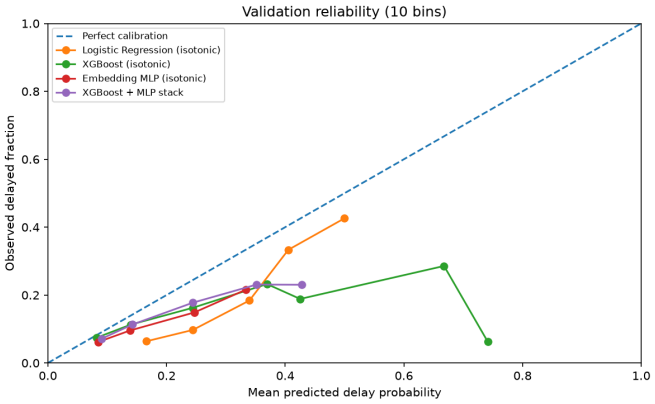


Fig. 15. Validation reliability for the calibrated AML models, calibrated embedding MLP, and hybrid. The diagonal indicates perfect calibration.

for the hybrid. Under the headline action-cost ratio of 0.25 and recovery of 0.8, all four policies incurred higher simulated costs than inaction. AP remains the primary ranking metric because trapezoidal PR area can be sensitive to interpolation at the endpoint when isotonic calibration creates large tied groups.

X. DISCUSSION AND LIMITATIONS

The results are deliberately unembellished. Discrimination is modest (ROC-AUC ≈ 0.60), the calibrated probabilities do not convincingly improve on a prevalence baseline in proper-score terms, and the between-model difference is not statistically established. These results describe the tested models and dataset; they do not establish a performance ceiling. Forecast weather, live operational conditions and upstream-aircraft state are potential additional signals whose value was not evaluated here. The gain-importance ranking (Fig. 13) describes this fitted model only and does not bound other pre-departure classifiers. The rescheduling analysis shows a narrow operating regime. Key limitations: (i) historical features guarantee prior-date ordering but not exact availability-time safety; (ii) the data cover a single year and only three New York origin airports, limiting generalisation; (iii) the calibration study is conditional on hyperparameters chosen over the full original training period and is not fully nested; (iv) the rescheduling cost model assumes homogeneous, independent per-flight costs with no network feedback; (v) delay prevalence shifts substantially across the chronological partitions (Fig. 1), so prevalence-dependent metrics are not directly comparable across windows; and (vi) the validation-only congestion ablation changed the aggregate ranking metrics only slightly in this dataset, which does not establish the features' value in a larger multi-airport setting. None of the results supports a production-readiness claim.

XI. CONCLUSION AND FUTURE WORK

We delivered a leakage-aware, calibrated and honestly evaluated baseline for pre-departure flight-delay prediction, together with an offline rescheduling what-if analysis grounded in explicit cost assumptions. XGBoost is the stronger descriptive candidate and the better basis for a cost-sensitive rescheduling rule, which nonetheless pays off only under cheap-intervention assumptions.

The embedding MLP and stacked hybrid were evaluated on the same validation flights as the classical models. The hybrid's Brier gain over calibrated XGBoost was 0.000054, with lower AP. The larger BTS on-time dataset offers a route to broader geographic coverage and a sterner test of generalisation.

REPRODUCIBILITY

The pipeline, models, evaluation and rescheduling analysis are implemented as a tested Python package with continuous integration for the classical and CPU PyTorch paths. The frozen evaluation records the dataset and artifact SHA-256 hashes and the exact dependency versions, so a reviewer can confirm that the reported numbers came from the committed data and code; the final test stage is additionally guarded so it cannot be silently re-run or re-selected.

TEAM CONTRIBUTIONS

In Phase 2, Lakshya integrated the dataset and audited historical availability. Ankit Raj Singh implemented categorical encoding and embedding tables. Samiksha Jangid implemented the embedding MLP and architecture tests. Rashmi Anand implemented training, tuning and checkpoint export. Shubhaang Kataruka implemented the AML–DL comparison and checkpoint audit.

REFERENCES

[1] Advanced Machine Learning lecture notes.
[2] Advanced Machine Learning lab notes.
[3] OpenAI, ChatGPT. Available: https://chatgpt.com/.
[4] Anthropic, Claude. Available: https://claude.ai/.
[5] U.S. Bureau of Transportation Statistics, "Airline On-Time Performance Data," TranStats database. [Online]. Available: https://www.transtats.bts.gov/.
[6] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.
[7] C. Guo and F. Berkhahn, "Entity Embeddings of Categorical Variables," *arXiv preprint arXiv:1604.06737*, 2016.
[8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?," *arXiv preprint arXiv:2207.08815*, 2022.
[9] R. Shwartz-Ziv and A. Armon, "Tabular Data: Deep Learning is Not All You Need," *arXiv preprint arXiv:2106.03253*, 2021.
[10] X. He et al., "Practical Lessons from Predicting Clicks on Ads at Facebook," in *Proceedings of the 8th International Workshop on Data Mining for Online Advertising (ADKDD)*, 2014, pp. 1–9.